Exercises

Design principles & Patterns

Exercise 1: Implementing the Singleton Pattern

```java
import java.util.*;
class Singleton {
    private static Singleton instance;
    Singleton() {
        System.out.println("Created a Singleton Class");
    } //Private constructor
    public static Singleton getinstance(){
        if (instance==null){
            instance=new Singleton();
        }
        return instance;
    }
}
```

```
PORTS
TERMINAL
PS P:\Cognizant_exercise> javac Singleton.java Main.java
PS P:\Cognizant_exercise> java Main
Created a Singleton Class
Both are same
PS P:\Cognizant_exercise>
```

Main Loading...

```java
public class Main{
    public static void main(String args[]){
        Singleton s1=Singleton.getinstance();
        Singleton s2=Singleton.getinstance();
        if(s1==s2){
            System.out.println("Both are same");
        }
    }
}
```

Exercise 2: Implementing the Factory Method Pattern

FactoryMethod.java

```java
import java.util.*;
interface Animal{
    void speak();
}
class Dog implements Animal{
    public void speak(){
        System.out.println("Woof");
    }
}
class Cat implements Animal{
    public void speak(){
        System.out.println("Meow");
    }
}
class FactoryMethod {
    public static void getFactory(String type){
        if(type=="Dog"){
            Dog d=new Dog();
        }
        else if(type=="Cat"){
            Cat c=new Cat();
        }
        else{
            System.out.println("Invalid Type");
```

```java
public class Main{
    public static void main(String args[]){x:
        Animal a=new Dog();
        a.speak();
        Animal b=new Cat();
        b.speak();
    }
}
```

```
AL   PORTS   QUERY RESULTS

TERMINAL
PS P:\Cognizant_exercise> javac FactoryMethod.java Main.java
PS P:\Cognizant_exercise> java Main
Woof
Meow
PS P:\Cognizant_exercise>
```

Exercise 2: E-commerce Platform Search Function

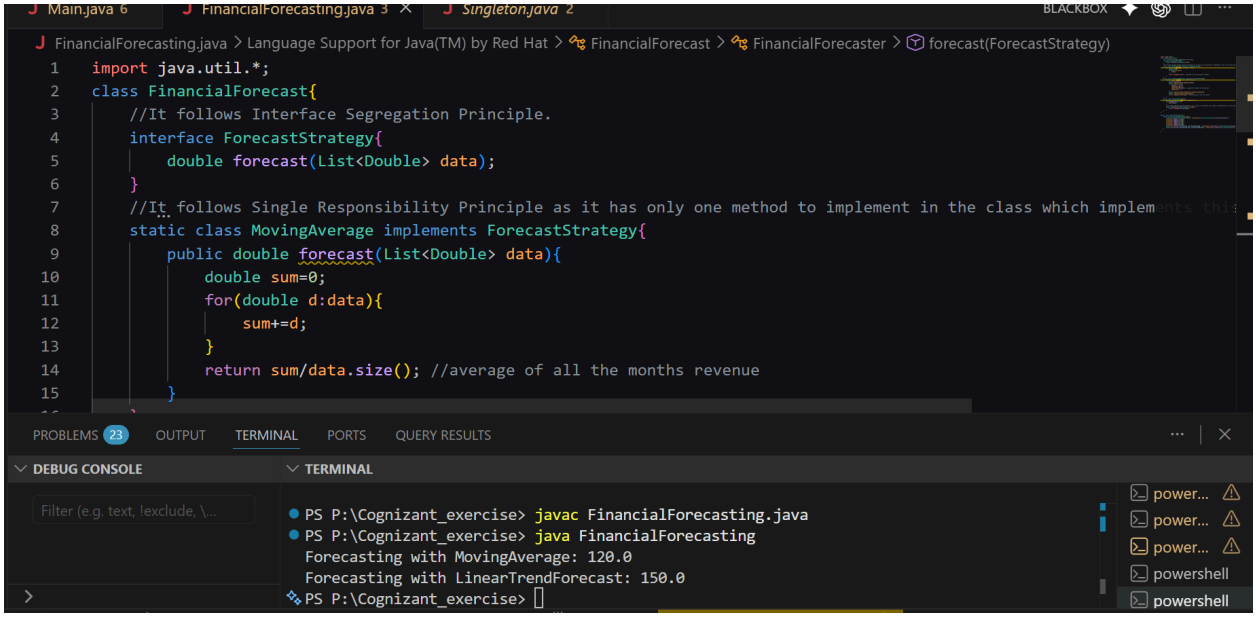
```java
import java.util.*;

class Ecommerce {
    static class Product{
        int id;
        String pname;
        String Category;
        double price;
        Product(int id,String pname,String Category,double price){
            this.id=id;
            this.pname=pname;
            this.Category=Category;
            this.price=price;
        }
        public String toString(){
            return id + "-" + pname + "-" + Category + "Rs."+price;
        }
    }
    //It follows Interface Segregation Principle.
    interface SearchStrategy{
        List<Product> search(List<Product> products, String query);
    }
    //It follows Single Responsibility Principle as it has only one method to implement in the class which implem
    static class KeywordSearch implements SearchStrategy{
```

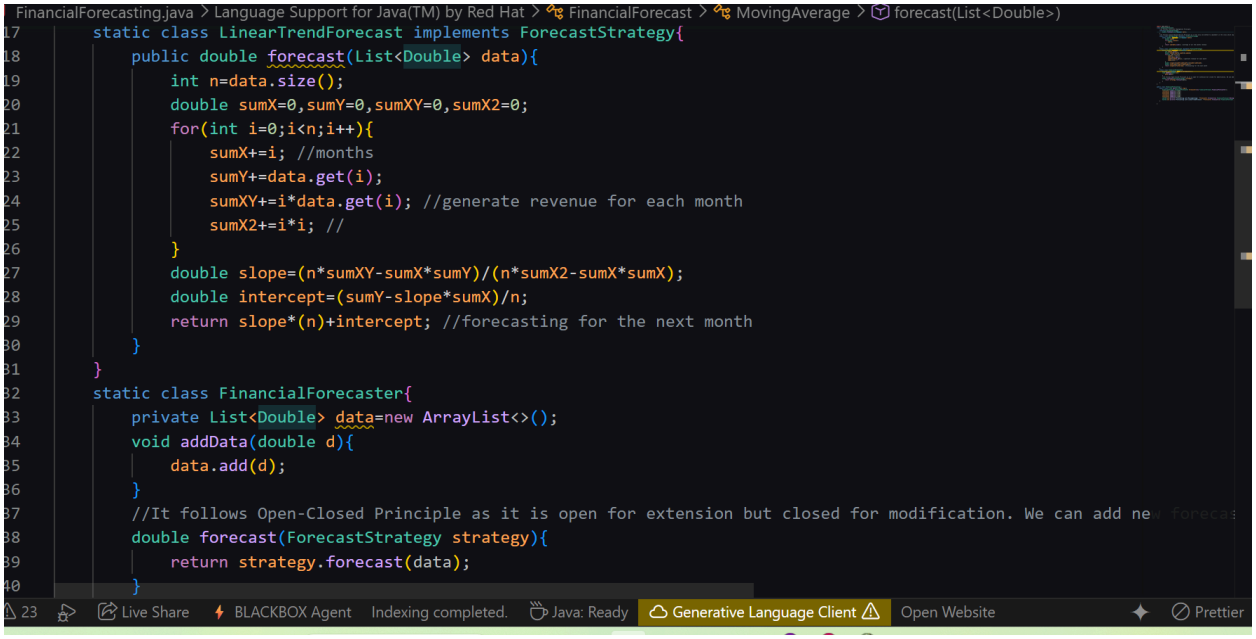
```java
static class KeywordSearch implements SearchStrategy{
    public List<Product> search(List<Product> products, String query){
        List<Product> result=new ArrayList<>();
        for(Product p:products){
            if(p.pname.toLowerCase().contains(query.toLowerCase())){
                result.add(p);
            }
        }
        return result;
    }
}
static class PriceSearch implements SearchStrategy{
    public List<Product> search(List<Product> products, String query){
        List<Product> result=new ArrayList<>();
        double price=Double.parseDouble(query);
        for(Product p:products){
            if(p.price<=price){
                result.add(p);
            }
        }
        return result;
    }
```

```
static class ProductCatalog{
    private List<Product> products=new ArrayList<>();
    private Map<Integer,Product> productMap=new HashMap<>();
    void addProduct(Product p){
        products.add(p);
        productMap.put(p.id,p);
    }
    Product getById(int id){
        return productMap.get(id);
    }
    List<Product> search(SearchStrategy strategy,String query){
        return strategy.search(products,query);
    }
}
```

```
public class Main{
    Run main | Debug main | Run | Debug
    public static void main(String[] args){
        Ecommerce.ProductCatalog catalog=new Ecommerce.ProductCatalog();
        catalog.addProduct(new Ecommerce.Product(id: 1,pname: "Laptop",Category: "Electronics",price: 50000));
        catalog.addProduct(new Ecommerce.Product(id: 2,pname: "Mobile",Category: "Electronics",price: 20000));
        catalog.addProduct(new Ecommerce.Product(id: 3,pname: "Shirt",Category: "Clothing",price: 1000));
        catalog.addProduct(new Ecommerce.Product(id: 4,pname: "Jeans",Category: "Clothing",price: 2000));
        catalog.addProduct(new Ecommerce.Product(id: 5,pname: "Shoes",Category: "Footwear",price: 3000));
        System.out.println(x: "Search by keyword 'Laptop':");
        List<Ecommerce.Product> result=catalog.search(new Ecommerce.KeywordSearch(),query: "Laptop");
        for(Ecommerce.Product p:result){
            System.out.println(p);
        }
        System.out.println(x: "Search by price <= 2000:");
        result=catalog.search(new Ecommerce.PriceSearch(),query: "2000");
        for(Ecommerce.Product p:result){
            System.out.println(p);
        }
    }
}
```

Exercise 7: Financial Forecasting





```
        }
    }
}

public class FinancialForecasting {
    Run main | Debug main | Run | Debug
    public static void main(String[] args){
        FinancialForecast.FinancialForecaster forecaster=new FinancialForecast.FinancialForecaster();
        forecaster.addData(d: 100);
        forecaster.addData(d: 110);
        forecaster.addData(d: 120);
        forecaster.addData(d: 130);
        forecaster.addData(d: 140);
        System.out.println("Forecasting with MovingAverage: "+forecaster.forecast(new FinancialForecast.MovingAve
        System.out.println("Forecasting with LinearTrendForecast: "+forecaster.forecast(new FinancialForecast.Lin
    }

}
```